Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, general style, even when parts of it could be computed during compilation.

Original program

```mls
fun dot(xs, ys, i) =
  if xs.length == i then 0
  else xs.(i) * ys.(i)
    + dot(xs, ys, i + 1)

fun dotWith3(v) =
  dot([1, 0, 2], v, 0)
```

can actually be written as

```mls
fun dotWith3(v) = v(0)
+ 2 * v(2)
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matching, the multiplication are all known at compile time. The residual program contains only the work that genuinely depends on the runtime input `v`.

Goal: let the programmer keep the abstract version, and have the compiler peel away the static layer automatically.

Multi-Stage Programming

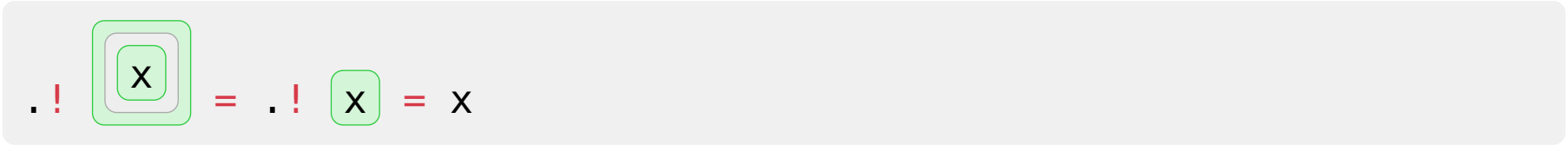
Well-known optimization technique [1]

```
1  fun power(n, x) = if n is
2    0 then .<1>.
3    else .<.~x * .~(power(n - 1, x))>.
4  fun power2 = .! .<(x => .~(power(2, .<x>.)))>.
```

In here, green annotates code to be executed in the next stage, and gray executed in the current stage.

Multi-Stage Programming

Rule for .!: evaluate until we the whole code fragment is in the next stage, then move it to the current stage.



During evaluation, we able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

```
1  x => power(2, x)                                                                    mls
2  x => if 2 is 0 then 1 else x * power(2 - 1, x)
3  x => x * power(1, x)
4  x => x * if 1 is 0 then 1 else x * power(1 - 1, x)
5  x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)
6  x => x * x * 1
7  fun power2 = x => x * x * 1
```

Class specialization

```
1  class B(val x) with
2    fun f() = ...
3  // ...
4  if x is
5    1 then new D1(1)
6    2 then new D2(2)
7
8  x.f()
```

```
…                    …
8   if x is                    mls
9     D1 then x.D1_f1()
10    D2 then x.D2_f1()
11    D3 then x.D3_f1()
```

Traditional Multi-Stage Programming tracks types.

Even if `x` has a known class shape, we cannot remove matching arms.

Specialization is done through typing, so we know that `x : B`, but we do not know which specific derived class of `B` is used.

Class specialization

```mls
data class Foo(x, y) extends Bar(x+1, y+1)

if new Foo(1, 2) is
  Bar(x, y) then ...
```

Under single inheritance, matching arms runs into problems. [2]

(Class splitting)

```mls
class Foo$1$2 extends Bar
class Bar$2$3
```

Approach in High Level

Start with an ordinary module:

```mls
module M with
  fun dot(xs, ys, i) =
    if xs.length == i then 0
    else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

Approach in High Level

Mark it as `staged`: the module now acts as a **code generator**: at compile time it specialises calls on known argument shapes and emits a residual module containing only the work that depends on runtime input.

```mls
staged module M with
  fun dot(xs, ys, i) =
    if xs.length == i then 0
    else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

Approach in High Level

After staging, We are left with an ordinary (non-staged) module containing just the residual program that the user can use

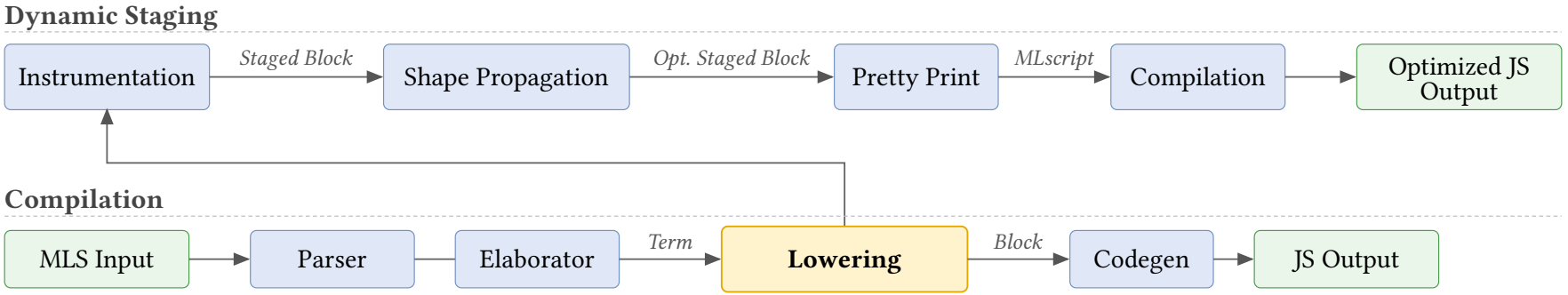
```mls
module M with
  fun dotWith3(v) = v.(0) + 2 * v.(2)
  ... with more specialized functions ...
```

Hence, our approach is a combination of **metaprogramming** (code generator) and **specialization** (generation of specialized functions).

MLscript Compiler

- Lexer/Parser: converts source code into AST
- Elaborator/Resolver: Deal with references
- **Lowering**: converts AST to a simplified format
- Codegen: Turn AST into executable code (JavaScript).

Dynamic Staging Recipe

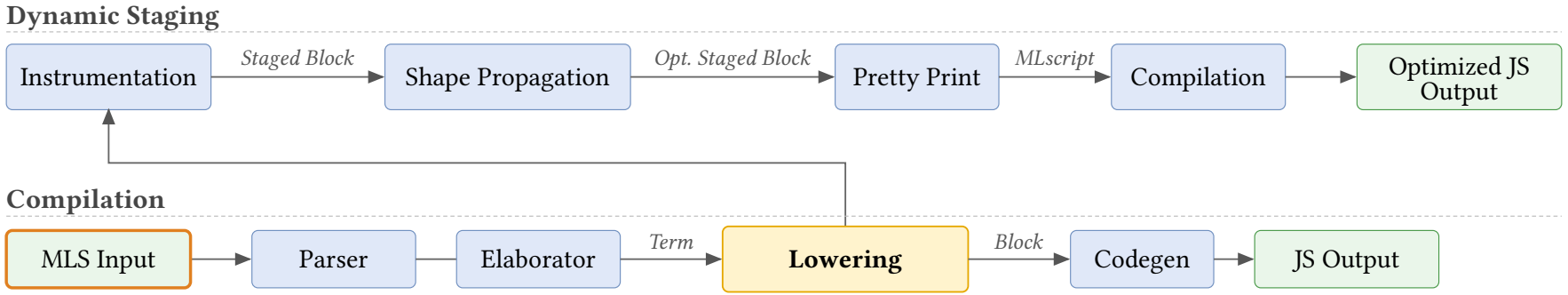


Two phases: Instrumentation / Shape Propagation

Lowering Pass

We implement staging a transformation from Scala Block ⇒ Scala Block.

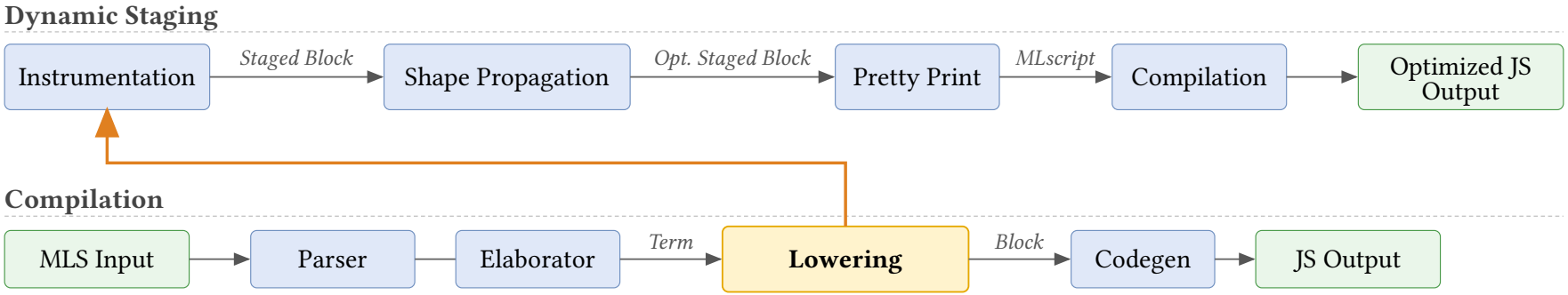
Dynamic Staging Recipe



Source (MLscript)

```mls
if C(0) is
  Int  then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

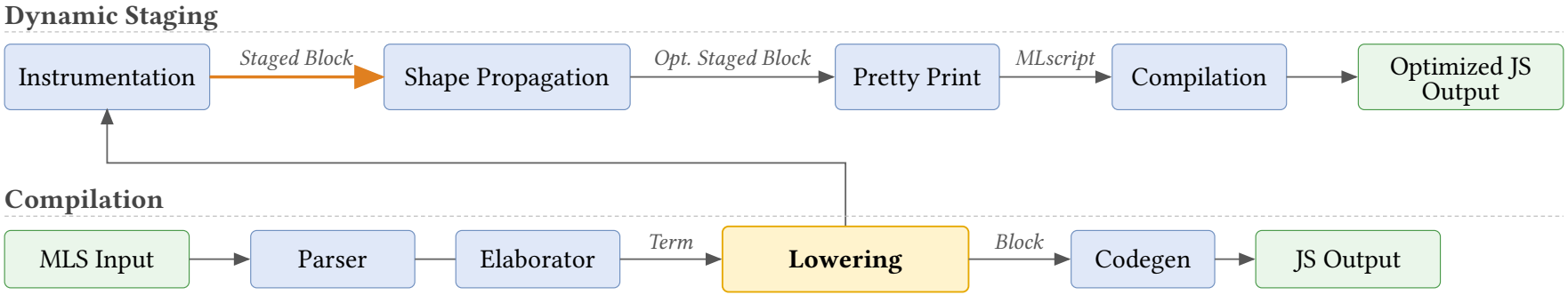
Dynamic Staging Recipe



Block

```scala
Scoped([scrut, n, arg, tmp],
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n,   Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ")"]))
  ], Ret("Unknown")))
```

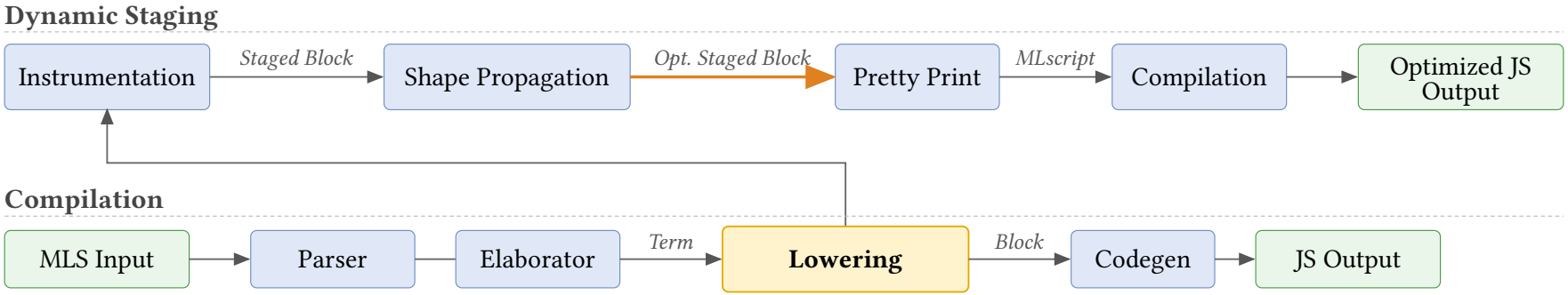
Dynamic Staging Recipe



Staged Block

```mls
Scoped([scrut, n, arg, tmp],
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n,   Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ")"]))
  ], Ret("Unknown")))
```

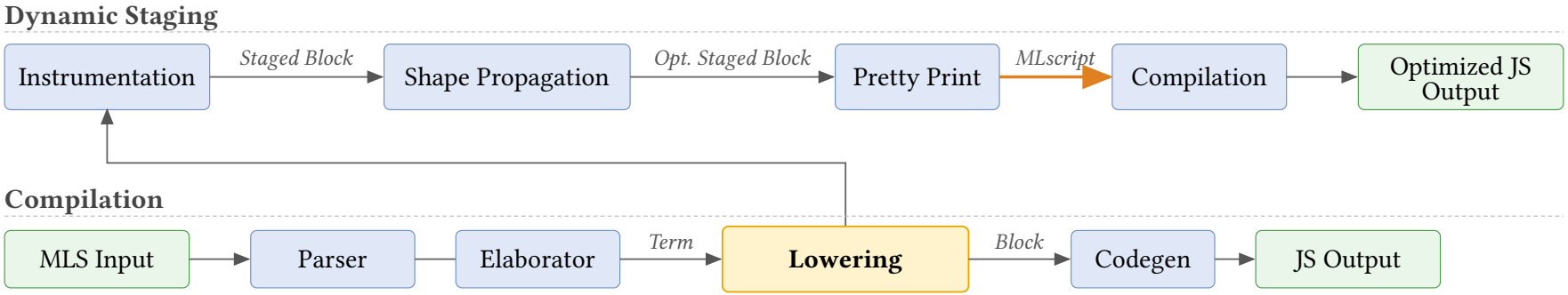
Dynamic Staging Recipe



Optimized Staged Block

```
Block.Ret("C(0)")                                    mls
```

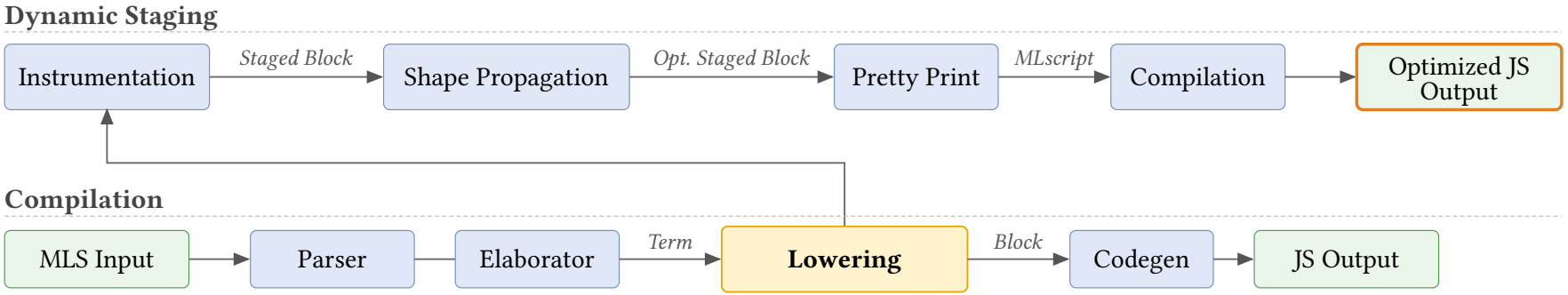
Dynamic Staging Recipe



Generated MLscript

```mls
"C(0)"
```

Dynamic Staging Recipe



Generated JS

```
"C(0)"                                                    JS JavaScript
```

Staging Block

```scala
case class Return(res, implct) extends Block
case class Match(scrut, arms, dflt, rest) extends Block
case class Assign(lhs, rhs, rest) extends Block
```

```mls
class Block with
  constructor
    Return(res, implct)
    Match(scrut, arms, dflt, rest)
    Assign(lhs, rhs, rest)
```

For any Scala Block data, we can recreate the same structure within MLscript which we called **Staged Block**.

Staging Block

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```scala
val pow = Symbol("pow"); val x = Symbol("x"); val n = Symbol("n")
val t1 = Symbol("tmp"); val t2 = Symbol("tmp")
val sub = Symbol("-"); val mul = Symbol("*")
FunDefn(pow, [x, n], Scoped(HashSet(t1, t2), Match(Ref(n),
  [[ Lit(0), Return(Lit(1)) ]],
  Assign(t1, Call(sub, [n, Lit(1)]),
    Assign(t2, Call(pow, [x, t1]),
      Return(Call(mul, [x, t2]))
    )
  ), End())
))
```

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging Block

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
let pow = Symbol("pow"); let x = Symbol("x"); let n = Symbol("n")
let t1 = Symbol("tmp"); let t2 = Symbol("tmp")
let sub = Symbol("-"); let mul = Symbol("*")
FunDefn(pow, [x, n], Scoped([t1, t2], Match(Ref(n),
  [[ Lit(0), Return(Lit(1)) ]],
  Assign(t1, Call(sub, [n, Lit(1)]),
    Assign(t2, Call(pow, [x, t1]),
      Return(Call(mul, [x, t2]))
    )
  ), End())
))
```

mls

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging Block

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

```
ValueLit(lit)                                    mls
```

Example: `1` $\mapsto$ `ValueLit(1)`

`Symbol(name)`

```
Symbol(name)                                     mls
```

Example:

- `x` $\mapsto$ `Symbol("x")`
- `c` $\mapsto$ ...?

This is not enough for staging symbols. We'll revisit this case later on.

Staging Block

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

```
ValueLit(lit)                                    mls
```

Example: `1` ↦ `ValueLit(1)`

`Value.Ref(sym)`

```
let l = sym                                      mls
ValueRef(l)
```

Example: `x` ↦ `ValueRef(Symbol("x"))`

`Select(qual, name)`

```
let q = qual                                     mls
let n = name
Select(q, n)
```

Example: `x.p` ↦ `Select(ValueRef(Symbol("x")), Symbol("p"))`

Staging Block

```
Tuple([x0, x1, ...])
```

```
let s0 = x0                                                    mls
let s1 = x1
                                    ...
Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging Symbols

- We need more information about the Symbol in the original stage

  Add reference to current value in the symbol

  `C.x`

```mls
let CSym = ClassSymbol("C", C)
Select(ValueRef(CSym), Symbol("x"))
```

  Add redirection within current stage to allow access for next stage

```mls
staged class A with
  fun f() = M.f()
  val redirect_M = M
```

Staging Symbols

- Uniqueness of Symbols

  We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For local symbols, we can maintain a map during staging to reuse a staged symbol.

```
x + x
```

```mls
let x = Symbol("x")
// let x1 = Symbol("x")
Call(ValueRef(Symbol("+")), [[ValueRef(x), ValueRef(x)]])
```

Staging Symbols

- Uniqueness of Symbols

  We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For class and module symbols, we need to cache and use first instance of a symbol within the staged code.

```
M.f()
```

```
let MSym = symbolMap.check(ModuleSymbol("M", M))                    mls
Call(Select(ValueRef(MSym), Symbol("f")), [[]])
```

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

mls

```
staged module Math with
  val funCache = new Map()
  val generatorMap = new Map([["pow", pow_gen]])
  fun pow_staged() = ...
  fun pow_gen(x, n) =
    specialize(funCache, "f", pow_staged, [x, n])
  fun propagate() =
    let dyn = ...
    pow_gen(dyn, dyn)
    sq_gen(dyn)
```

Shape Definition

We focus on tracking the shape of the Paths and Results.

$$s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \bot \mid s \cup s$$

```mls
fun f(p, cond) =
  let a = 42
  let b = new C(p, 2)
  let c = if cond then [1, 2, 3] else C(1, 2)
  let d = if b is C then 0 else 1
```

Shape Definition

Block in BNF

Symbol x, y, z *Literal* ι

Function definition $\mathbf{fun}\ f(\overline{x_i}^n) = b$ *Plain class definition* $\mathbf{class}\ C\left(\overline{\mathbf{val}\ n}\right)$

Staged module definition $\mathbf{staged\ module}\ M\ \mathbf{with\ fun}\ f(\overline{x_i}^n) = b$

$$
\begin{array}{rrcl}
\textit{Path} & p & ::= & \iota \mid x \mid p.n \\
\textit{Result} & r & ::= & p \mid f(\overline{p}) \mid \mathbf{new}\ C(\overline{p}) \mid [\overline{p}] \\
\textit{Match pattern} & \pi & ::= & \iota \mid C \mid []^n \mid \_ \\
\textit{Match arm} & \kappa & ::= & \pi\ \mathbf{then}\ b \\
\textit{Non-tail block} & B & ::= & \varepsilon \mid \mathbf{if}\ p\ \mathbf{is}\ \overline{\kappa}; B \mid x = r; B \mid \mathbf{let}\ x; B \\
\textit{Block} & b & ::= & \mathbf{return}\ r \mid \mathbf{end} \mid B; b \\
\textit{Shape} & s & ::= & [\![\iota]\!] \mid \mathbf{dyn} \mid [\![[\overline{s}]]\!] \mid [\![C(\overline{n : s})]\!] \mid \bot \mid s \cup s \\
\textit{Shape context} & \Gamma & ::= & \varepsilon \mid \Gamma, p \mapsto s \quad (p \neq x)
\end{array}
$$

Shape Propagation (Example)

A **Shape** captures what we statically know about a value at compile time.

$$s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \bot \mid s \cup s$$

We write ⟦s⟧ to denote the shape of an expression, distinguishing it from actual values.

```
fun f(p, cond) =                                              mls
  let a = 42                                                  ⟦42⟧
  let b = C(p, 2)                                             ⟦C(dyn, 2)⟧
  let c = if cond then [1, 2] else C(1, 2)                    ⟦[1,2] ∪ C(1,2)⟧
  let d = if b is C then 0 else 1                             ⟦0⟧
```

Shape Propagation (Example)

A First Taste

A tiny program, propagating shapes by hand:

```mls
staged module M with
  fun add1(x) = x + 1
  fun test()  = add1(2)
```

Walking through `test()`:

1. Argument `2` has shape $[\![2]\!]$ (a literal).
2. Specialise `add1` with `x` $\mapsto [\![2]\!]$ inside the body, `x` looks up $[\![2]\!]$ in the context.
3. `x + 1` folds: $[\![2]\!]$ `+ 1` $\rightarrow [\![3]\!]$.
4. Cache the result as `add1_Lit2() = 3`; rewrite the call site `add1(2)` to `add1_Lit2()`.

Final shape of `test()`: $[\![3]\!]$. Body collapses to a constant.

Shape Propagation (Formalization)

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

We evaluate paths p to their shapes s.

$$\frac{}{\Gamma \vdash \iota \Rightarrow [\![\iota]\!]}$$

Example

```
42                                mls
```

$\Gamma = \varepsilon$
$\varepsilon \vdash 42 \Rightarrow [\![42]\!]$

$$\frac{(p \mapsto s) \in \Gamma \quad s \neq \bot}{\Gamma \vdash p \Rightarrow s}$$

Example

```
x                                 mls
```

$\Gamma = x \mapsto [\![C(n : [\![1]\!])]\!]$
$\Gamma \vdash x \Rightarrow [\![C(n : [\![1]\!])]\!]$

$$\frac{p \notin \mathrm{dom}(\Gamma) \quad \Gamma \vdash p \Rightarrow s}{\Gamma \vdash p.n \Rightarrow \mathrm{sel}(s, [\![n]\!])}$$

Example

```
x.n                               mls
```

$\Gamma = x \mapsto [\![C(n : [\![1]\!])]\!]$
$\Gamma \vdash x.n \Rightarrow [\![1]\!]$

Reminder: $p ::= \iota \mid x \mid p.n$

Shape Propagation (Formalization)

Shape Selection (`sel`)

sel computes the shape of a selection.

$$\mathrm{sel}(\llbracket C(\overline{n_i : s_i}) \rrbracket, \llbracket n_i \rrbracket) = s_i \qquad \mathrm{sel}(\mathbf{dyn}, \llbracket n_i \rrbracket) = \mathbf{dyn}$$

$$\mathrm{sel}(\llbracket [\overline{s_i}^{\,i \in 0..n}] \rrbracket, \llbracket i \rrbracket) = s_i \qquad \mathrm{sel}(s_1 \cup s_2, s) = \mathrm{sel}(s_1, s) \cup \mathrm{sel}(s_2, s)$$

$$\mathrm{sel}(s_1, s_2) = \mathbf{err} \quad \text{otherwise}$$

Example

$$\begin{aligned}\mathrm{sel}(\llbracket [1,2] \rrbracket \cup \llbracket [2,3,4] \rrbracket, \llbracket 1 \rrbracket) &= \mathrm{sel}(\llbracket [1,2] \rrbracket, \llbracket 1 \rrbracket) \cup \mathrm{sel}(\llbracket [2,3,4] \rrbracket, \llbracket 1 \rrbracket) \\ &= \llbracket 2 \rrbracket \cup \llbracket 3 \rrbracket\end{aligned}$$

Shape Propagation (Formalization)

Shape of Result (sor: $\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

We optimize block and track shapes simultaneously: r becomes r' with shape s.

$$\frac{\Gamma \vdash p \Rightarrow s}{\Gamma \vdash p \rightsquigarrow p \Rightarrow s}$$

Example

```
x
```
mls

$\Gamma = x \mapsto [\![1]\!]$
$\Gamma \vdash x \rightsquigarrow x \Rightarrow [\![1]\!]$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash [\overline{p_i}^n] \rightsquigarrow [\overline{p_i}^n] \Rightarrow [\![[\overline{s_i}^n]]\!]}$$

Example

```
[x, y]
```
mls

$\Gamma = x \mapsto [\![1]\!], y \mapsto [\![2]\!]$
$\Gamma \vdash [x, y] \rightsquigarrow [x, y] \Rightarrow [\![[[\![1]\!], [\![2]\!]]]\!]$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash \textbf{new}\ C(\overline{p_i}) \rightsquigarrow \textbf{new}\ C(\overline{p_i}) \Rightarrow [\![C(\overline{n_i : s_i})]\!]}$$

Example

```
new Pair(x, y)
```
mls

$\Gamma = x \mapsto [\![1]\!], y \mapsto [\![2]\!]$
$\Gamma \vdash \textbf{new}\ \mathrm{Pair}(x, y) \Rightarrow [\![\mathrm{Pair}(a : [\![1]\!], b : [\![2]\!])]\!]$

Reminder: $r ::= p \mid [\overline{p}] \mid \textbf{new}\ C(\overline{p}) \mid f(\overline{p})$

Shape Propagation (Formalization)

Shape of Result (`sor`) — Staged Application

When f is **staged**, we recursively specialise its body on the argument shapes.

$$\frac{f \quad \text{is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}\,(\overline{p}, \overline{s}) = (r', s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r' \Rightarrow s}$$

Example

```mls
staged module Staged with
  fun pyth(x, y) = x * x + y * y
  fun test() = pyth(2, 3)
```

Specialise $\text{pyth}_{\text{gen}}\,([\![2]\!], [\![3]\!]) \rightarrow$ produces `pyth_Lit2_Lit3() = 13`, shape $[\![13]\!]$.
Conclusion: $\varepsilon \vdash \text{pyth}(2,3) \rightsquigarrow 13 \Rightarrow [\![13]\!]$

Reminder: $r ::= p \mid [\overline{p}] \mid \textbf{new}\ C(\overline{p}) \mid f(\overline{p})$

Shape Propagation (Formalization)

Shape of Result (`sor`) — After Specialisation

After propagation, the call site `pyth(2, 3)` is rewritten to its cached variant.

$$\frac{f \quad \text{is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}\,(\overline{p}, \overline{s}) = (r', s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r' \Rightarrow s}$$

Example

```
staged module Staged with                                    mls
  fun pyth_Lit2_Lit3() = 13
  fun test() = pyth_Lit2_Lit3()
```

Reminder: $r ::= p \mid [\overline{p}] \mid \textbf{new}\ C(\overline{p}) \mid f(\overline{p})$

Shape Propagation (Formalization)

Shape of Result (sor) — Non-Staged Application

When f is **non-staged**, we cannot peek into its body. Two sub-cases.

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \forall i.\ \mathrm{static}(s_i) \quad f_{\mathrm{imp}}\left(\overline{\mathrm{valOf}(s)}\right) = (r, s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r \Rightarrow s}$$

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \exists i.\ \neg\ \mathrm{static}(s_i)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow f(\overline{p_i}) \Rightarrow \mathbf{dyn}}$$

Example

```
// fun sq(x) = x * x  (non-staged, pure)                mls
NonStaged.sq(2)
```

All args static; evaluate $\mathrm{sq}(2) = 4$.
$\varepsilon \vdash \mathrm{sq}(2) \rightsquigarrow 4 \Rightarrow [\![4]\!]$

Example

```
// pyth's body, x has dynamic shape                     mls
NonStaged.sq(x)
```

$\Gamma = x \mapsto \mathbf{dyn}$; cannot evaluate, emit code.
$\Gamma \vdash \mathrm{sq}(x) \rightsquigarrow \mathrm{sq}(x) \Rightarrow \mathbf{dyn}$

Shape Propagation (Formalization)

Static Shape

$$\text{static}([\![\iota]\!]) = \textbf{true} \qquad \text{static}([\![[\overline{s_i}^n]]\!]) = \bigwedge_i \text{static}(s_i)$$

$$\text{static}([\![C(\overline{n_i : s_i})]\!]) = \bigwedge_i \text{static}(s_i) \qquad \text{static}(s_1 \cup s_2) = \text{static}(s_1) \wedge \text{static}(s_2)$$

$$\text{static}(\textbf{dyn}) = \textbf{false} \qquad \text{static}(\bot) = \textbf{false}$$

Remark. $\text{static}(s)$ returns true iff no subshape of s contains $\textbf{dyn}$.

Shape Propagation (Pattern Matching)

Before giving the formal rules, let's give a simple trace of shape propagation for pattern matching.

```mls
class C(val n)
staged module Simple with
  fun f(x) =
    let y
    if x is C then y = x.n
    else y = 0
    y + 1
  fun test()     = f(C(2))
  fun test2(dyn) = f(C(dyn))
  fun test3()    = f(0)
```

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto$ ⟦`C(n: 2)`⟧.

```
fun f(x) =                                    mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` $\longmapsto$ ⟦`C(n: 2)`⟧

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\mapsto [\![$`C(n: 2)`$]\!]$.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` $\mapsto [\![$`C(n: 2)`$]\!]$

`y` $\mapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\mapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\mapsto [\![$`C(n: 2)`$]\!]$.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` $\mapsto [\![$`C(n: 2)`$]\!]$

`y` $\mapsto [\![$`2`$]\!]$

1. `let y`: extend ctx with `y` $\mapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`$[\![$`C(n: 2)`$]\!]$`, n)` = $[\![$`2`$]\!]$, so `y` $\mapsto [\![$`2`$]\!]$

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$.

```
fun f(x) =                    mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$

`y` $\longmapsto$ $[\![$`2`$]\!]$

1. `let y`: extend ctx with `y` $\longmapsto$ $[\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`$[\![$`C(n: 2)`$]\!]$`, n)` = $[\![$`2`$]\!]$, so `y` $\longmapsto$ $[\![$`2`$]\!]$
4. `y + 1` folds to $[\![$`3`$]\!]$, then `Return 3`

Cached as `f_C_Lit2`; body folds entirely to the constant `3`.

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```mls
fun f(x) =
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦⊥⟧

1. `let y: y` ⟼ ⟦⊥⟧

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦⊥⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C)` = ⟦`C(n: dyn)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C) =` ⟦`C(n: dyn)`⟧ so `then` is viable; `rest =` ⟦⊥⟧ so `else` is dead
3. In `then`: `sop(x.n) = sel(`⟦`C(n: dyn)`⟧`, n) =` ⟦`dyn`⟧

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦`C(n: dyn)`⟧`, C)` = ⟦`C(n: dyn)`⟧ so `then` is viable; `rest` = ⟦⊥⟧ so `else` is dead
3. In `then`: `sop(x.n)` = `sel(`⟦`C(n: dyn)`⟧`, n)` = ⟦`dyn`⟧
4. `y + 1`: cannot fold (⟦`dyn`⟧ `+ 1`); emit code, result ⟦`dyn`⟧

Cached as `f_C_Dyn`; body keeps the `y = x.n` assignment.

Shape Propagation (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦`0`⟧ (literal).

Specialise `f` with `x` ⟼ ⟦`0`⟧.

ctx

`x` ⟼ ⟦`0`⟧

```mls
fun f(x) =
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

Shape Propagation (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦`0`⟧ (literal).

Specialise `f` with `x` ⟼ ⟦`0`⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦`0`⟧

`y` ⟼ ⟦⊥⟧

1. `let y:` `y` ⟼ ⟦⊥⟧

Shape Propagation (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).

Specialise `f` with `x` ⟼ ⟦0⟧.

```
fun f(x) =                    mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦⊥⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable

Shape Propagation (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).

Specialise `f` with `x` ⟼ ⟦0⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦0⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable
3. In `else`: `y` ⟼ ⟦0⟧

Shape Propagation (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧ (literal).
Specialise `f` with `x` ⟼ ⟦0⟧.

```
fun f(x) =                                  mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx
`x` ⟼ ⟦0⟧
`y` ⟼ ⟦0⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable
3. In `else`: `y` ⟼ ⟦0⟧
4. `y + 1` folds to ⟦1⟧, then `Return 1`

Cached as `f_Lit0`.

Shape Propagation (Pattern Matching)

Resulting Cache

After all three calls, the staged module contains:

```mls
module Simple withs
  fun f_C_Dyn(x) =
    let y
    y = x.n
    y + 1                    // dyn case: code is kept
  fun f_C_Lit2(x) = 3        // fully specialised
  fun f_Lit0()    = 1        // fully specialised
  fun test()      = 3
  fun test2(dyn)  = f_C_Dyn(C$If2(dyn))
  fun test3()     = 1
```

Pattern Matching (Formalization)

Branch Filter (`filter`)

filter removes impossible shapes from the branches of a pattern match.

$$\begin{aligned}
\text{filter}(s, _) &= s \\
\text{filter}([\![[\overline{s_i}^n]]\!], []^n) &= [\![[\overline{s_i}^n]]\!] \\
\text{filter}(\mathbf{dyn}, \pi) &= \text{silh}(\pi) \\
\text{filter}(s, \pi) &= \bot \quad \text{otherwise}
\end{aligned}$$

$$\begin{aligned}
\text{filter}([\![\iota_1]\!], \iota_2) &= [\![\iota_1]\!] \quad \text{if} \quad \iota_1 = \iota_2 \\
\text{filter}([\![C(\overline{n_i : s_i}^m)]\!], C) &= [\![C(\overline{n_i : s_i}^m)]\!] \\
\text{filter}(s_1 \cup s_2, \pi) &= \text{filter}(s_1, \pi) \cup \text{filter}(s_2, \pi)
\end{aligned}$$

Pattern Matching (Formalization)

Branch Remainder (`rest`)

rest removes shapes already covered by a match pattern.

$$\begin{aligned}
\text{rest}(s, _) &= \bot & \text{rest}([\![\iota_1]\!], \iota_2) &= \bot \quad \text{if} \quad \iota_1 = \iota_2 \\
\text{rest}([\![[\overline{s_i}^n]]\!], []^n) &= \bot & \text{rest}([\![C(\overline{n_i : s_i}^m)]\!], C) &= \bot \\
\text{rest}(\mathbf{dyn}, \pi) &= \mathbf{dyn} \quad \text{if} \quad \pi \neq _ & \text{rest}(s_1 \cup s_2, \pi) &= \text{rest}(s_1, \pi) \cup \text{rest}(s_2, \pi) \\
\text{rest}(s, \pi) &= s \quad \text{otherwise}
\end{aligned}$$

Pattern Matching (Formalization)

Dynamic Dispatching

When a method is called on a value with a **union shape**, we split by class and dispatch separately.

```
staged class B1(val y) with  fun call(x) = x + 2 + y
staged class B2(val y) with  fun call(x) = x + y
staged module M with
  fun twice(f, x) = f.call(f.call(x))
  fun pick(x, y, b) = if b then x else y
  fun f(b) =
    let m = pick(new B1(2), new B2(3), b)
    twice(m, 5)
```

Pattern Matching (Formalization)

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![$`dyn`$]\!]$.

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new
  B2(3), b)
  twice(m, 5)
```

ctx

`b` $\longmapsto [\![$`dyn`$]\!]$

Pattern Matching (Formalization)

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![\texttt{dyn}]\!]$.

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new B2(3), b)
  twice(m, 5)
```

ctx

`b` $\longmapsto [\![\texttt{dyn}]\!]$

`m` $\longmapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\longmapsto [\![\bot]\!]$

Pattern Matching (Formalization)

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![\texttt{dyn}]\!]$.

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new B2(3), b)
  twice(m, 5)
```

ctx

`b` $\longmapsto [\![\texttt{dyn}]\!]$

`m` $\longmapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\longmapsto [\![\bot]\!]$
2. Call `pick(new B1(2), new B2(3), b)`: argument shapes are $[\![\texttt{B1(2)}]\!]$, $[\![\texttt{B2(3)}]\!]$, $[\![\texttt{dyn}]\!]$. Recurse into `pick` with a fresh ctx
   $\rightarrow$ **go to Trace B**

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =                mls
  if b then x
  else y
```

ctx

fresh ctx for `pick`

$\texttt{x} \longmapsto [\![\texttt{B1(2)}]\!]$

$\texttt{y} \longmapsto [\![\texttt{B2(3)}]\!]$

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =                    mls
  if b then x
  else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =          mls
  if b then x
  else y
```

ctx

fresh ctx for `pick`

$\texttt{x} \longmapsto [\![\texttt{B1(2)}]\!]$

$\texttt{y} \longmapsto [\![\texttt{B2(3)}]\!]$

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

1. `if b`: `b` is $[\![\texttt{dyn}]\!]$, so both branches viable
2. `then` arm returns `x`, shape = $[\![\texttt{B1(2)}]\!]$

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =          mls
  if b then x
  else y
```

ctx

fresh ctx for `pick`

`x` ⟼ ⟦`B1(2)`⟧

`y` ⟼ ⟦`B2(3)`⟧

`b` ⟼ ⟦`dyn`⟧

1. `if b`: `b` is ⟦`dyn`⟧, so both branches viable
2. `then` arm returns `x`, shape = ⟦`B1(2)`⟧
3. `else` arm returns `y`, shape = ⟦`B2(3)`⟧

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =                mls
  if b then x
  else y
```

ctx

fresh ctx for `pick`

$\texttt{x} \longmapsto [\![\texttt{B1(2)}]\!]$

$\texttt{y} \longmapsto [\![\texttt{B2(3)}]\!]$

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

1. `if b`: `b` is $[\![\texttt{dyn}]\!]$, so both branches viable
2. `then` arm returns `x`, shape = $[\![\texttt{B1(2)}]\!]$
3. `else` arm returns `y`, shape = $[\![\texttt{B2(3)}]\!]$
4. Result shape: $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$. Cached as `pick1`. Return to Trace A.

Pattern Matching (Formalization)

Trace A (continued): back in `f`

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new B2(3), b)
  twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

Pattern Matching (Formalization)

Trace A (continued): back in `f`

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new B2(3), b)
  twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

1. Bind `m` to `pick`'s returned shape = $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

Pattern Matching (Formalization)

Trace A (continued): back in `f`

```
fun f(b) =                                    mls
  let m = pick(new B1(2), new
  B2(3), b)
  twice(m, 5)
```

ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

1. Bind `m` to `pick`'s returned shape = $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$
2. `twice(m, 5)`: `m` has **union shape** → **go to Trace C** and specialise as `twice1`

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =                    mls
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

ctx

$f \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$x \longmapsto [\![5]\!]$

Recall:

```
class B1(y) with                     mls
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =                    mls
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

Recall:

```
class B1(y) with                     mls
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\bot]\!]$

1. `let tmp`: extend ctx with $\texttt{tmp} \longmapsto [\![\bot]\!]$

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =                    mls
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

Recall:

```
class B1(y) with                     mls
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

ctx

$\texttt{f} \mapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \mapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \mapsto [\![\texttt{\{9, 8\}}]\!]$

1. `let tmp`: extend ctx with $\texttt{tmp} \mapsto [\![\bot]\!]$
2. `f.call(x)` on union $\rightarrow$ **split by class**:
   - $\texttt{f} = [\![\texttt{B1(2)}]\!]$: cache `f.call1()` $\rightarrow [\![\texttt{9}]\!]$
   - $\texttt{f} = [\![\texttt{B2(3)}]\!]$: cache `f.call1()` $\rightarrow [\![\texttt{8}]\!]$

Pattern Matching (Formalization)

Trace C (continued): second call

```
fun twice(f, x) =                    mls
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

Recall:

```
class B1(y) with fun
call(x) = x + 2 + y                  mls
class B2(y) with fun call(x) =
x + y
```

ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\texttt{\{9, 8\}}]\!]$

3. `f.call(tmp)` with `tmp` in $[\![\texttt{\{9, 8\}}]\!]$ (now $[\![\texttt{dyn}]\!]$): split again, body kept as code
   - `f` = $[\![\texttt{B1}]\!]$: cache `f.call2(x) = x + 2 + 2`
   - `f` = $[\![\texttt{B2}]\!]$: cache `f.call2(x) = x + 3`

Pattern Matching (Formalization)

Resulting Cache

After propagation, `M` and the staged classes contain:

```
module M with                                    mls
  fun f(b) =
    let m, tmp1, tmp2
    tmp1 = new B1(2)
    tmp2 = new B2(3)
    m = pick1(tmp1, tmp2, b)
    twice1(m)
  fun pick1(x, y, b) =
    if b then new B1(2) else new B2(3)
```

```
  fun twice1(f) =
    let tmp
    if f is
      B1 then tmp = f.call1()
      B2 then tmp = f.call1()
    if f is
      B1 then f.call2(tmp)
      B2 then f.call2(tmp)
```

Pattern Matching (Formalization)

mls

```
class B1(y) with
  fun call1() = 9
  fun call2(x) =
    let tmp
    tmp = x + 2
    tmp + 2

class B2(y) with
  fun call1() = 8
  fun call2(x) = x + 3
```

Printing Staged Block

After all the specialized functions are completed, we need to write the functions to a new file.

```mls
staged module M with
  val funCache = new Map([
    ["f1", ...],
    ["f2", ...],
    ["g1", ...]
  ])
```

```mls
module M with
  fun f1(x, y) = ...
  fun f2() = ...
  fun g1() = ...
```

Printing Staged Block

This is a similar process to staging, but done in reverse.

As before, we need extra care when handling symbols.

Model-View-Projection transformation

basically we're pretty good at partially evaluating matrices ^w^.

Time: about 2x (from 2s to 1s, eh...)

Code size: don't worry about it, we'll just do lifting

Bibliography

[1] W. Taha, "A gentle introduction to multi-stage programming," in *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, 2004, pp. 30–50.

[2] A. Shali and W. R. Cook, "Hybrid partial evaluation," in *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*, 2011, pp. 375–390.